

?? Building an MCP Text2SQL Server and Client - From Scratch

?? Building an MCP Text2SQL Server and Client - From Scratch

In this notebook, we're going to build a complete MCP-based Text2SQL agent - piece by piece, function by function.

By the end, you'll have a system where:

- A user asks a question in plain English
- An LLM figures out which database tools to call
- Those tools run via the Model Context Protocol (MCP)
- The LLM gets the results back and gives a human-readable answer

?? This notebook is a teaching walkthrough, not a runnable Colab. We're here to understand how each piece works and why it exists. To actually run the code, use the companion repository.

Section 1 - What Are We Building?

Before we write any code, let's get the big picture clear.

We're building three things:

1. An MCP Server - A process that exposes database tools (list tables, get schema, run queries)
2. An MCP Client - A wrapper that spawns the server and talks to it over stdio
3. An Agent Loop - The glue that connects an OpenAI LLM to the MCP client

Here's how they fit together:

The LLM never touches the database directly. It asks the MCP server to do it - through well-defined tools with schemas.

The Three Tools

Our agent needs exactly three tools. This matches the workflow from the Agents Colab:

The agent calls them in order: discover ? inspect ? query. Let's build each one.

Project Layout

Heres the folder structure were working toward. Dont worry about creating these files yet - well build each one as we go.

Code

```
# Target project structure
project_layout = """
mcp-text2sql/
?
??? data/
?   ??? sample.db           ? SQLite database (generated)
?
??? src/
?   ??? mcp_server.py       ? The MCP server (exposes 3 tools)
?   ??? mcp_client.py       ? The MCP client (connects to server)
?   ??? agent.py            ? The agent (LLM + MCP client)
?
??? scripts/
?   ??? setup_db.py         ? Creates the sample database
?
??? .env                    ? API keys (git-ignored)
??? pyproject.toml          ? Dependencies
"""
print(project_layout)
```

A few things to notice:

- * has three files, each with one job. The server knows about the database. The client knows about MCP transport. The agent knows about the LLM. They dont peek into each others internals.
- * is generated, not committed. The setup script creates it fresh every time.
- * holds your OpenAI key - it never goes into Git.

Section 2 - Setting Up the Sample Database

Before we can build tools that talk to a database, we need a database. We'll use SQLite - it's built right into Python, so there's nothing to install.

Let's start by defining where the database file will live.

Code

```
import sqlite3
from pathlib import Path
DB_PATH = Path("data/sample.db")
```

Simple enough. We use `Path` so the path works on Mac, Linux, and Windows.

Now let's create the database directory if it doesn't exist, and open a connection.

Code

```
DB_PATH.parent.mkdir(parents=True, exist_ok=True)
conn = sqlite3.connect(str(DB_PATH))
cur = conn.cursor()
```

The `employees` Table

This is the first table. It matches the one from the Agents Colab - 8 employees across 3 departments and 3 cities. Enough to support questions like "What's the average salary by department?" or "List all employees in Berlin."

Code

```

cur.execute("""
CREATE TABLE employees (
id          INTEGER PRIMARY KEY,
name        TEXT      NOT NULL,
department  TEXT      NOT NULL,
salary      REAL      NOT NULL,
city        TEXT      NOT NULL
);
""")
cur.executemany(
"INSERT INTO employees VALUES (?, ?, ?, ?, ?)",
[
(1, "Alice", "Engineering", 95000, "Berlin"),
(2, "Bob",   "Engineering", 88000, "Berlin"),
(3, "Carol", "Marketing",   72000, "Prague"),
(4, "David", "Marketing",   68000, "Prague"),
(5, "Eve",   "Engineering", 102000, "London"),
(6, "Frank", "Sales",       61000, "London"),
(7, "Grace", "Sales",       59000, "Berlin"),
(8, "Heidi", "Engineering", 97000, "Prague"),
],
)

```

That gives us 8 rows. Four engineers, two in marketing, two in sales. Salaries range from 59k to 102k. The department and city columns let us do GROUP BY and WHERE filtering.

The `customers` Table

This one is designed to answer the classic Colab question: How many customers pay in EUR?

We have 10 customers across 6 countries with 5 different currencies. So the agent cant just count all rows - it has to filter by .

Code

```
cur.execute("""
CREATE TABLE customers (
  id          INTEGER PRIMARY KEY,
  name        TEXT      NOT NULL,
  country     TEXT      NOT NULL,
  currency    TEXT      NOT NULL,
  balance     REAL       NOT NULL
);
""")
cur.executemany(
  "INSERT INTO customers VALUES (?, ?, ?, ?, ?)",
  [
    (1, "Acme GmbH", "Germany", "EUR", 15400.50),
    (2, "Prague Widgets", "Czech Republic", "CZK", 342000.00),
    (3, "Berlin Tech", "Germany", "EUR", 88200.00),
    (4, "London Analytics", "UK", "GBP", 45000.00),
    (5, "CZ Solutions", "Czech Republic", "CZK", 198000.00),
    (6, "Euro Supplies", "France", "EUR", 23100.75),
    (7, "Nordic Data", "Sweden", "SEK", 67000.00),
    (8, "Praha Services", "Czech Republic", "CZK", 410000.00),
    (9, "Milan Corp", "Italy", "EUR", 54200.00),
    (10, "Swiss Holdings", "Switzerland", "CHF", 120000.00),
  ],
)
```

4 of those 10 customers pay in EUR. That's the answer the agent should arrive at.

The `orders` Table

This is where things get interesting. The table has a foreign key to . That means the agent will need to do JOIN queries to answer questions like Show all orders from German customers.

Code

```
cur.execute("""
CREATE TABLE orders (
id            INTEGER PRIMARY KEY,
customer_id  INTEGER NOT NULL,
product      TEXT     NOT NULL,
amount       REAL     NOT NULL,
order_date   TEXT     NOT NULL,
FOREIGN KEY (customer_id) REFERENCES customers(id)
);
""")
cur.executemany(
"INSERT INTO orders VALUES (?, ?, ?, ?, ?)",
[
(1, 1, "Widget A", 2500.00, "2024-11-15"),
(2, 1, "Widget B", 1800.00, "2025-01-10"),
(3, 2, "Gadget X", 5400.00, "2025-02-01"),
(4, 3, "Widget A", 3200.00, "2025-01-20"),
(5, 4, "Service Pro", 9000.00, "2025-03-05"),
(6, 5, "Gadget X", 2700.00, "2024-12-22"),
(7, 6, "Widget B", 1100.00, "2025-02-14"),
(8, 9, "Service Pro", 6500.00, "2025-01-30"),
(9, 3, "Gadget Y", 4100.00, "2025-03-12"),
(10, 7, "Widget A", 1900.00, "2025-02-28"),
],
)
conn.commit()
conn.close()
```

And were done with the database. Three tables, 28 rows total.

To answer a JOIN question, the agent will need to:

1. Discover both and tables
2. Inspect both schemas to find the foreign key
3. Write a JOIN query

Thats a 4-tool-call conversation - exactly the kind of multi-step reasoning we want to see from the agent.

Section 3 - Implementing the Tool Functions

Now for the fun part. Were going to build the three tool functions that our MCP server will expose.

We'll write them as plain Python functions first - no MCP, no decorators, no protocol stuff. Just functions that take inputs and return outputs. We can test them directly. Later, we'll wrap them with MCP decorators.

Every tool follows the same pattern:

1. Open a database connection
2. Run a query
3. Return the result
4. Always close the connection (even if something crashes)

The `\_get\_connection` Helper

All three tools need a database connection. Instead of repeating the same connection code three times, lets pull it into a small helper function.

It also checks that the database file actually exists - and gives a helpful error message if it doesnt.

Code

```
import sqlite3
from pathlib import Path
DB_PATH = "data/sample.db"
def _get_connection() -> sqlite3.Connection:
    """Return a new SQLite connection. Raises if DB not found."""
    if not Path(DB_PATH).exists():
        raise FileNotFoundError(
            f"Database not found at {DB_PATH}. "
            "Run 'python scripts/setup_db.py' first."
        )
    return sqlite3.connect(DB_PATH)
```

Why do we create a fresh connection every time instead of keeping one open globally?

SQLite connections are cheap to create. A fresh connection per call avoids problems with stale state, threading, and leaked connections. The pattern in each tool makes sure the connection always gets closed - even if the query throws an error.

Tool 1 - The `list\_tables` Function

This is the first tool the agent will call for any question. It queries SQLite's internal table to discover all the user-created tables.

Without this tool, the agent would have to guess table names - which means hallucinated SQL. We don't want that.

Code

```
def list_tables() -> list[str]:
    """Get all table names from the SQLite database."""
    conn = _get_connection()
    try:
        cur = conn.cursor()
        cur.execute(
            "SELECT name FROM sqlite_master WHERE type='table' ORDER BY name;"
        )
        tables = [row[0] for row in cur.fetchall()]
        return tables
    finally:
        conn.close()
```

Lets break this down:

- * We filter by so we skip internal SQLite objects like indexes and views.
- * We so the output is always the same - no randomness.
- * The return type is - clean and simple for the LLM to read.
- * The ensures runs even if the query fails.

Lets test it real quick:

Code

```
# Quick test
tables = list_tables()
print(tables)
# Expected: ['customers', 'employees', 'orders']
```

Three tables. Exactly what we put in. The agent will see this list and decide which table(s) to inspect next.

Tool 2 - The `get\_table\_schema` Function

OK, so the agent now knows theres a table. But it doesnt know what columns it has. It cant just wing it and write - that column doesnt exist.

So the next step is to inspect the schema. SQLite has a built-in way to do this: .

But heres the important thing - the parameter comes from the LLM. And LLMs are non-deterministic. So we need input validation before we use it in a query.

Code

```

import re
def get_table_schema(table_name: str) -> list[dict]:
    """Get column names and types for a specific table."""
    # Prevent SQL injection via table name
    if not re.match(r"^[A-Za-z_][A-Za-z0-9_]*$", table_name):
        raise ValueError(f"Invalid table name: {table_name!r}")
    conn = _get_connection()
    try:
        cur = conn.cursor()
        cur.execute(f"PRAGMA table_info(`{table_name}`);")
        rows = cur.fetchall()
        if not rows:
            raise ValueError(
                f"Table '{table_name}' not found in the database."
            )
        columns = [
            {
                "name": row[1],
                "type": row[2],
                "nullable": not row[3],
                "primary_key": bool(row[5]),
            }
            for row in rows
        ]
        return columns
    finally:
        conn.close()

```

Lets talk about that regex for a second:

It only allows valid SQL identifiers - letters, numbers, underscores, starting with a letter or underscore. Why do we care?

Because without it, a hallucinated LLM input like `would get injected straight into the query. The regex catches this and raises an error instead.`

This is defense-in-depth. The LLM shouldnt send malicious input. But we dont trust it - tools are the boundary between the non-deterministic agent and our real system, and they must validate everything.

Lets test it:

Code

```
# Test: valid table
schema = get_table_schema("customers")
for col in schema:
    print(f" {col['name']:12s} {col['type']:10s} pk={col['primary_key']}")
# Expected:
# id          INTEGER    pk=True
# name        TEXT       pk=False
# country     TEXT       pk=False
# currency   TEXT       pk=False
# balance     REAL       pk=False
```

Code

```
# Test: SQL injection attempt - should be blocked
try:
    get_table_schema("employees; DROP TABLE employees")
except ValueError as e:
    print(f"Blocked: {e}")
# Expected: Blocked: Invalid table name: 'employees; DROP TABLE employees'
```

Good. The valid call returns column metadata. The injection attempt gets caught at the door. The database is never touched.

Tool 3 - The `execute\_sql` Function

This is the most powerful tool - and the most dangerous. It runs actual SQL against the database. That's why it has two layers of safety:

1. Keyword blocking - We reject any query that starts with or contains , , , , , , or .
2. Error handling - If the SQL itself fails (bad syntax, wrong column name), we catch the error and return it as a string instead of crashing. This is important because the agent can read the error, figure out what went wrong, and try again with a corrected query.

Code

```

def execute_sql(query: str) -> str:
    """Execute a SQL SELECT query and return results as a formatted string."""
    # ?? Safety: reject anything that isn't a SELECT ??????????
    normalized = query.strip().upper()
    forbidden = [
        "INSERT", "UPDATE", "DELETE", "DROP",
        "ALTER", "CREATE", "TRUNCATE", "REPLACE",
    ]
    for keyword in forbidden:
        if (normalized.startswith(keyword)
            or f" {keyword} " in f" {normalized} "):
            raise ValueError(
                f"Only SELECT queries are allowed. "
                f"Detected forbidden keyword: {keyword}"
            )
    # ?? Run the query ?????????????????????????????????????????
    conn = _get_connection()
    try:
        cur = conn.cursor()
        cur.execute(query)
        columns = [
            desc[0] for desc in cur.description
        ] if cur.description else []
        rows = cur.fetchall()
        # ?? Format as a readable table ?????????????????????????
        if not rows:
            return "(no rows returned)"
        header = " | ".join(columns)
        separator = "-+-".join(
            "-" * max(len(col), 8) for col in columns
        )
        data_rows = [
            " | ".join(str(val) for val in row)
            for row in rows
        ]
        result = f"{header}\n{separator}\n" + "\n".join(data_rows)
        return result
    except sqlite3.Error as e:
        return f"SQL Error: {e}"
    finally:
        conn.close()

```

A couple of things to notice here.

Why return a string instead of a list of dicts? Because the LLM reads the tool output as text. A formatted table like `table` is much easier for the model to interpret than nested JSON. It can just read it and summarize.

Why catch errors instead of crashing? In a ReAct loop, a failed query isn't the end of the world - it's information. When the agent sees `ValueError`, it can think: Hmm, the column is probably called `currency`, not `curr`. Let me check the schema. That self-correction is a core part of agentic behavior.

Lets test both the happy path and the safety checks:

Code

```
# Test 1: Valid SELECT
print("?? Valid query ??")
result = execute_sql(
    "SELECT COUNT(*) AS eur_count FROM customers WHERE currency = 'EUR'"
)
print(result)
# Expected:
# eur_count
# -----
# 4
```

Code

```
# Test 2: Dangerous query - should be rejected
print("?? Dangerous query ??")
try:
    execute_sql("DELETE FROM employees WHERE id = 1")
except ValueError as e:
    print(f"Blocked: {e}")
# Expected:
# Blocked: Only SELECT queries are allowed. Detected forbidden keyword: DELETE
```

Code

```
# Test 3: A JOIN query - this is what the agent will write for
# "Show orders from German customers"
print("?? JOIN query ??")
result = execute_sql(
    "SELECT c.name, o.product, o.amount "
    "FROM orders o "
    "JOIN customers c ON o.customer_id = c.id "
    "WHERE c.country = 'Germany'"
)
print(result)
```

All three tools are working. We can discover tables, inspect schemas, and run queries safely. Now lets wrap them in an MCP server.

Section 4 - Creating the MCP Server

Weve got our three functions. Now we need to make them available over the Model Context Protocol. The MCP servers job is to:

1. Advertise the tools - their names, descriptions, and parameter schemas
2. Handle incoming tool calls from any MCP client
3. Communicate over a transport - in our case, stdio (standard input/output)

We'll use the `Server` class from the Python SDK. If you've used Flask or FastAPI, the pattern will feel very familiar - its decorator-based.

Creating the Server Instance

Lets start with the very first thing we need: a server object. This is one line of code.

Code

```
from mcp.server.fastmcp import FastMCP
mcp = FastMCP("Text2SQL-MCP", log_level="INFO")
```

Thats it. We now have a server called . It can register tools, resources, and prompts. But right now it has nothing in it - its an empty shell.

Lets fix that.

Registering Tool 1 - `list\_tables`

To turn our plain function into an MCP tool, we add the decorator. The decorator does three things behind the scenes:

1. Registers the function in the servers tool registry
2. Generates a JSON Schema from the function signature
3. Sets up a handler so when a client calls , this function runs

The is the most important part. Its what the LLM reads to decide when to use this tool. A vague description like database stuff would confuse the model. A clear one like Get all table names - call this first guides its reasoning.

Code


```

from pydantic import Field
@mcp.tool(
    name="list_tables",
    description=(
        "Get all table names from the SQLite database. "
        "Call this first to discover which tables are available "
        "before writing any queries."
    ),
)
def list_tables() -> list[str]:
    conn = _get_connection()
    try:
        cur = conn.cursor()
        cur.execute(
            "SELECT name FROM sqlite_master "
            "WHERE type='table' ORDER BY name;"
        )
        tables = [row[0] for row in cur.fetchall()]
        return tables
    finally:
        conn.close()

```

What did the decorator generate? Behind the scenes, the MCP SDK created this JSON Schema:

The client (and the LLM) never see our Python code. They only see this schema. That's the whole point of MCP - the protocol is the contract between server and client.

Registering Tool 2 - `get\_table\_schema`

This tool has a parameter: . Notice how we use Pydantics to describe it. That description becomes part of the JSON Schema - so the LLM knows exactly what value to pass.

Code

```
@mcp.tool(
    name="get_table_schema",
    description=(
        "Get column names and types for a specific table. "
        "Use this after identifying the relevant table "
        "with list_tables."
    ),
)
def get_table_schema(
    table_name: str = Field(
        description="The exact name of the table to inspect"
    ),
) -> list[dict]:
    if not re.match(r"^[A-Za-z_][A-Za-z0-9_]*$", table_name):
        raise ValueError(f"Invalid table name: {table_name!r}")
    conn = _get_connection()
    try:
        cur = conn.cursor()
        cur.execute(f"PRAGMA table_info(`{table_name}`);")
        rows = cur.fetchall()
        if not rows:
            raise ValueError(
                f"Table '{table_name}' not found in the database."
            )
        columns = [
            {
                "name": row[1], "type": row[2],
                "nullable": not row[3],
                "primary_key": bool(row[5]),
            }
            for row in rows
        ]
        return columns
    finally:
        conn.close()
```

The generated schema for this tool now includes the parameter:

When the LLM decides to call this tool, it generates JSON like . MCP deserializes that and passes it to our function. We never parse JSON ourselves.

Registering Tool 3 - `execute\_sql`

The big one. Notice the description explicitly says ONLY SELECT - were telling the LLM upfront whats allowed so it doesnt waste iterations generating a DELETE that will just get blocked.

Code

```

@mcp.tool(
    name="execute_sql",
    description=(
        "Execute a SQL SELECT query against the database and "
        "return results as rows. ONLY SELECT queries are "
        "allowed - no INSERT, UPDATE, DELETE, DROP, or ALTER."
    ),
)
def execute_sql(
    query: str = Field(
        description="A valid SQL SELECT query to execute"
    ),
) -> str:
    normalized = query.strip().upper()
    forbidden = [
        "INSERT", "UPDATE", "DELETE", "DROP",
        "ALTER", "CREATE", "TRUNCATE", "REPLACE",
    ]
    for keyword in forbidden:
        if (normalized.startswith(keyword)
            or f" {keyword} " in f" {normalized} "):
            raise ValueError(
                f"Only SELECT queries are allowed. "
                f"Detected forbidden keyword: {keyword}"
            )
    conn = _get_connection()
    try:
        cur = conn.cursor()
        cur.execute(query)
        columns = [
            desc[0] for desc in cur.description
        ] if cur.description else []
        rows = cur.fetchall()
        if not rows:
            return "(no rows returned)"
        header = " | ".join(columns)
        separator = "-+-".join(
            "-" * max(len(col), 8) for col in columns
        )
        data_rows = [
            " | ".join(str(val) for val in row)
            for row in rows
        ]
        return f"{header}\n{separator}\n" + "\n".join(data_rows)
    except sqlite3.Error as e:
        return f"SQL Error: {e}"
    finally:
        conn.close()

```

We now have all three tools registered on the server. The MCP SDK knows about them, has their schemas, and can dispatch calls to them.

Lets add a couple of bonus features.

Bonus: Adding MCP Resources

MCP supports resources in addition to tools. Resources are read-only data endpoints - think GET endpoints in a REST API. Theyre useful for pre-loading context.

We'll expose the same data as our first two tools, but through the resource protocol:

Code

```
@mcp.resource("db://tables", mime_type="application/json")
def resource_list_tables() -> list[str]:
    """Resource: table names as JSON."""
    return list_tables()
@mcp.resource(
    "db://schema/{table_name}", mime_type="application/json"
)
def resource_table_schema(table_name: str) -> list[dict]:
    """Resource: column info for a table as JSON."""
    return get_table_schema(table_name)
```

Whats the difference between tools and resources?

In our demo, tools are the main interface. Resources are a bonus showing the full MCP surface area.

Bonus: Adding an MCP Prompt

MCP also supports prompts - reusable prompt templates that the client can request from the server. This is useful when you want the server to define the persona, rather than hardcoding it in the client.

Code

```
from mcp.server.fastmcp.prompts import base
@mcp.prompt(
    name="text2sql_analyst",
    description="Generate a system prompt for a Text2SQL analyst.",
)
def text2sql_analyst_prompt(
    question: str = Field(
        description="The user's natural-language question"
    ),
) -> list[base.Message]:
    prompt = f"""You are a Senior SQL Developer and Data Analyst.
    Follow this workflow:
    1. List tables with list_tables.
    2. Get schema with get_table_schema.
    3. Write a SQLite SELECT query.
    4. Execute with execute_sql.
    RULES: Only use confirmed names. Never modify data. Use SQLite syntax.
    OUTPUT: Show SQL, result, and a brief summary.
    USER QUESTION: {question}"""
    return [base.UserMessage(prompt)]
```

The Server Entry Point

Last piece: we tell the server to start listening. We use `-` - the server reads JSON-RPC messages from stdin and writes responses to stdout. The client will spawn this as a subprocess and pipe data through.

No HTTP server, no ports, no CORS - just stdin and stdout.

Code

```
if __name__ == "__main__":  
    mcp.run(transport="stdio")
```

That's the entire server. Let's recap what we have:

- * A server instance
- * 3 registered tools with schemas
- * 2 resources (bonus)
- * 1 prompt template (bonus)
- * A stdio entry point

The server file is ~150 lines of code. Most of it is the tool logic - the MCP wiring is just decorators and one call.

Now let's build the client that talks to it.

Section 5 - Building the MCP Client

The client's job is pretty straightforward:

1. Spawn the MCP server as a subprocess
2. Connect to it over stdio
3. Discover what tools are available
4. Call tools on behalf of the agent
5. Clean up when we're done

We'll wrap all of this in a class that supports Python's `contextmanager` pattern - so cleanup happens automatically even if something goes wrong.

The Client Constructor

Let's start with the `Client` class. It stores the command to run the server, and sets up an `AsyncExitStack`.

What's `AsyncExitStack`? It tracks all the async resources we open - the subprocess, the transport, the session - and closes them all in reverse order when we're done. Think of it as a stack of blocks that we build up one at a time.

Code

```
import asyncio
import json
from contextlib import AsyncExitStack
from typing import Any, Optional
from mcp import ClientSession, StdioServerParameters, types
from mcp.client.stdio import stdio_client
class MCPClient:
def __init__(
self,
command: str,
args: list[str],
env: Optional[dict] = None,
) -> None:
self._command = command
self._args = args
self._env = env
self._session: Optional[ClientSession] = None
self._exit_stack = AsyncExitStack()
```

Nothing has happened yet. Weve just stored the config. The actual connection comes next.

The `connect` Method

This is where the magic happens. In four steps:

1. Create - tells MCP how to spawn the server
2. Call - spawns the subprocess, gives us read/write streams
3. Create a - the MCP protocol handler over those streams
4. Call - performs the MCP handshake (version negotiation)

Code

```
async def connect(self) -> None:
    server_params = StdioServerParameters(
        command=self._command,
        args=self._args,
        env=self._env,
    )
    # Spawn the server process, get stdio streams
    stdio_transport = await self._exit_stack.enter_async_context(
        stdio_client(server_params)
    )
    read_stream, write_stream = stdio_transport
    # Open a ClientSession over those streams
    self._session = await self._exit_stack.enter_async_context(
        ClientSession(read_stream, write_stream)
    )
    # MCP handshake
    await self._session.initialize()
```

After `connect` completes, the client and server have agreed on a protocol version. The client can now ask `What tools do you have?` and the server will answer.

Notice how both `stdio_client` and `ClientSession` are registered with the `exit_stack`. When cleanup happens, they'll be closed in reverse order - `session` first, then the subprocess.

Making It Work with `async with`

We want to use the client like this:

To make that work, we need `AsyncContextManager` and `AsyncExitStack`:

Code

```
async def cleanup(self) -> None:
    await self._exit_stack.aclose()
    self._session = None
    async def __aenter__(self) -> "MCPCClient":
        await self.connect()
        return self
    async def __aexit__(self, exc_type, exc_val, exc_tb) -> None:
        await self.cleanup()
```

The `cleanup` method which closes the `session` - that shuts down the session and kills the subprocess. This happens even if an exception was thrown inside the `__aenter__` block. No resource leaks.

Session Property

A small guard - if someone tries to use the client before connecting, they get a clear error instead of a cryptic crash:

Code

```
@property
def session(self) -> ClientSession:
    if self._session is None:
        raise ConnectionError(
            "Client not connected. "
            "Use 'async with MCPCClient(...)' or call connect()."
        )
    return self._session
```

Tool Operations

These are the methods the agent will actually call. They're thin wrappers around :

Code

```
async def list_tools(self) -> list[types.Tool]:
    """Ask the server what tools are available."""
    result = await self.session.list_tools()
    return result.tools
async def call_tool(
    self, tool_name: str, tool_input: dict
) -> types.CallToolResult:
    """Call a specific tool with the given arguments."""
    return await self.session.call_tool(tool_name, tool_input)
```

returns a list of objects. Each one has:

- like or
- what the tool does (the LLM reads this)
- JSON Schema describing the tool's parameters

The agent will convert these to OpenAI's format and send them with every LLM call. That's how the LLM knows what tools exist and how to call them.

Resource and Prompt Operations

For completeness, the client also supports reading resources and prompts:

Code

```
async def list_prompts(self) -> list[types.Prompt]:
    result = await self.session.list_prompts()
    return result.prompts
async def get_prompt(
    self, prompt_name: str, args: dict[str, str]
) -> Any:
    result = await self.session.get_prompt(prompt_name, args)
    return result.messages
async def read_resource(self, uri: str) -> Any:
    from pydantic import AnyUrl
    result = await self.session.read_resource(AnyUrl(uri))
    resource = result.contents[0]
    if isinstance(resource, types.TextResourceContents):
        if resource.mimeType == "application/json":
            return json.loads(resource.text)
        return resource.text
    return resource
```

Thats the complete client class. Lets verify it can connect to our server.

Quick Self-Test

Heres a quick test that spawns the server, connects, discovers tools, and makes one call:

Code

```

async def _self_test() -> None:
    async with MCPClient(
        command="python",
        args=["src/mcp_server.py"],
    ) as client:
        tools = await client.list_tools()
        print(f"? Connected. Tools: {[t.name for t in tools]}")
        result = await client.call_tool("list_tables", {})
        print(f"? Tables: {result.content[0].text}")
    # asyncio.run(_self_test())
    # Expected output:
    # ? Connected. Tools: ['list_tables', 'get_table_schema', 'execute_sql']
    # ? Tables: ["customers", "employees", "orders"]

```

The client connects, discovers 3 tools, calls , and gets the result back - all over stdio pipes. No HTTP, no ports, no network. Just two processes talking to each other through stdin/stdout.

Now lets wire in the LLM.

Section 6 - The Agent Loop

This is where everything comes together. The agent implements the ReAct pattern:

The agent doesnt know in advance how many tool calls itll make. The LLM decides at each step - thats what makes this agentic instead of a fixed pipeline.

The System Prompt (Persona)

First, we define the agents persona. This is the Component 1 from the Agents Colab. It tells the LLM who it is, what workflow to follow, and what rules to obey.

The system prompt is the single most impactful piece of prompt engineering in the entire system.

Code

```
SYSTEM_PROMPT = """You are a Senior SQL Developer and Data Analyst.
INSTRUCTIONS:
You generate and execute SQL queries against a SQLite database.
Follow this workflow for every user question:
1. List all available tables using the list_tables tool.
2. Get the schema (columns) of the relevant table(s)
   using get_table_schema.
3. Write a syntactically correct SQLite SELECT query.
4. Execute the query using execute_sql and return the results.
RULES:
- ONLY use table and column names confirmed by the schema tools.
- NEVER guess or invent table or column names.
- NEVER run DELETE, DROP, UPDATE, INSERT, or any data-modifying SQL.
- If the question is unclear, state your assumptions first.
- If a query fails, analyze the error and retry with
  a corrected query.
- Use SQLite syntax (e.g., || for concatenation, not CONCAT).
OUTPUT FORMAT:
Always respond with:
1. The SQL query you generated
2. The query result
3. A short natural-language summary of the answer
"""
```

Notice the explicit workflow in the prompt: 1. List tables. 2. Get schema. 3. Write SQL. 4. Execute. Without this, the LLM might skip steps - guess a table name, invent a column, write broken SQL. The prompt keeps it disciplined while still letting it adapt (like calling on multiple tables for a JOIN).

Converting MCP Tool Schemas to OpenAI Format

The MCP server gives us tool schemas in MCP format. OpenAIs API expects them in its own format. This function bridges the two:

Code

```
from typing import Any
def _mcp_tools_to_openai_schema(tools) -> list[dict[str, Any]]:
    """Convert MCP tool definitions to OpenAI function-calling format."""
    openai_tools = []
    for tool in tools:
        schema = {
            "type": "function",
            "function": {
                "name": tool.name,
                "description": tool.description or "",
                "parameters": (
                    tool.inputSchema
                    if tool.inputSchema
                    else {"type": "object", "properties": {}}
                ),
            },
        }
        openai_tools.append(schema)
    return openai_tools
```

This is the MCP value proposition in a nutshell. The server defines tools once. The client discovers them. This converter adapts them for OpenAI - but you could write the same thing for Anthropic Claude, Google Gemini, or any other provider. The server code never changes.

The ReAct Loop

Now for the heart of the agent. Lets build it step by step.

Step 1 - Initialization: Connect to the MCP server, discover the tools, convert them to OpenAI format, start the conversation with our system prompt.

Code

```
from openai import OpenAI
OPENAI_MODEL = "gpt-4o-mini"
MAX_ITERATIONS = 15 # Safety limit
async def run_agent_loop(client, openai_client) -> None:
    # Discover tools from the MCP server
    mcp_tools = await client.list_tools()
    openai_tools = _mcp_tools_to_openai_schema(mcp_tools)
    tool_names = [t.name for t in mcp_tools]
    print(f"?? Text2SQL Agent")
    print(f"Tools: {tool_names}")
    # Start conversation with the persona
    messages = [{"role": "system", "content": SYSTEM_PROMPT}]
```

Step 2 - User input: Get the question, add it to the message history.

Code


```
# Inside run_agent_loop, continued...
while True:
    user_input = input("You: ").strip()
    if user_input.lower() in ("quit", "exit"):
        print("? Goodbye!")
        break
    messages.append({"role": "user", "content": user_input})
```

Step 3 - The inner loop: This is where the agent thinks. We call the LLM, check if it wants to use a tool, execute the tool via MCP, feed the result back, and repeat - until the LLM gives a final text answer.

Code

```

# Inside the while True loop, continued...
for iteration in range(MAX_ITERATIONS):
    # Call the LLM with tool schemas
    response = openai_client.chat.completions.create(
        model=OPENAI_MODEL,
        temperature=0.0,
        messages=messages,
        tools=openai_tools,
        tool_choice="auto",
    )
    assistant_msg = response.choices[0].message
    # ?? CASE A: Final text answer ??????????????????
    if not assistant_msg.tool_calls:
        print(f"\n? Agent:\n{assistant_msg.content}\n")
        messages.append({
            "role": "assistant",
            "content": assistant_msg.content,
        })
        break
    # ?? CASE B: Tool call(s) requested ??????????????
    messages.append(assistant_msg.model_dump())
    for tool_call in assistant_msg.tool_calls:
        fn_name = tool_call.function.name
        fn_args = json.loads(tool_call.function.arguments)
        print(
            f" ? [{iteration+1}] "
            f"{fn_name}({json.dumps(fn_args)})"
        )
    # Forward to MCP server
    try:
        result = await client.call_tool(fn_name, fn_args)
        tool_output = (
            result.content[0].text
            if result.content
            else "(empty)"
        )
    except Exception as e:
        tool_output = f"Error: {e}"
    # Feed result back to the LLM
    messages.append({
        "role": "tool",
        "tool_call_id": tool_call.id,
        "content": tool_output,
    })

```

Lets trace through exactly what happens for How many customers pay in EUR?:

...

Iteration 1:

LLM thinks ? I should discover the tables first

Returns: tool_call = list_tables({})

? MCP server runs list_tables()

? Returns: [customers, employees, orders]

Iteration 2:

LLM thinks ? I need the customers schema

Returns: tool_call = get_table_schema({table_name: customers})

? MCP server runs PRAGMA table_info(customers)

? Returns: [id, name, country, currency, balance]

Iteration 3:

LLM thinks ? Now I can write the SQL

Returns: tool_call = execute_sql({

query: SELECT COUNT(*) AS eur_count FROM customers WHERE currency = EUR
})

? MCP server runs the query

? Returns: eur_count\n\n4

Iteration 4:

LLM thinks ? I have all the info, time to answer

Returns: final text = There are 4 customers who pay in EUR.

...

4 LLM calls. 3 MCP tool calls. About \$0.002 total cost.

The Entry Point

Finally, we wire it all together. creates the OpenAI client, spawns the MCP server as a subprocess, and starts the agent loop:

Code

```

import os
from dotenv import load_dotenv
load_dotenv()
async def main() -> None:
    api_key = os.getenv("OPENAI_API_KEY", "")
    if not api_key:
        print("? OPENAI_API_KEY not set.")
        return
    openai_client = OpenAI(api_key=api_key)
    async with MCPClient(
        command="python",
        args=["src/mcp_server.py"],
    ) as mcp_client:
        await run_agent_loop(mcp_client, openai_client)
    # asyncio.run(main())

```

Heres what happens when you run this:

1. spawns as a child process
2. The MCP handshake happens over stdin/stdout pipes
3. calls - discovers 3 tools
4. The user types a question
5. The ReAct loop runs: LLM ? tool call ? result ? LLM ? ? answer
6. When the user types , kills the subprocess

The whole system is two processes talking through pipes. No server running in the background, no network, no docker.

Section 7 - End-to-End: Seeing the Full Picture

The Complete Data Flow

Lets trace one question through the entire system:

What the `messages` List Looks Like After

If you could peek inside the `list` after answering one question, youd see something like this:

```
```python
messages = [
Persona (always first)
{"role": "system", "content": "You are a Senior SQL Developer"},

The question
{"role": "user", "content": "How many customers pay in EUR?"},
LLM decided to call list_tables
{"role": "assistant", "tool_calls": [{name: "list_tables", ...}]},
{"role": "tool", "content": '["customers", "employees", "orders"]'},
LLM decided to call get_table_schema
{"role": "assistant", "tool_calls": [{name: "get_table_schema", ...}]},
{"role": "tool", "content": '["id", {name: "currency"}, ...]'},
LLM decided to call execute_sql
{"role": "assistant", "tool_calls": [{name: "execute_sql", ...}]},
{"role": "tool", "content": "eur_count\n-----\n4"},
LLM's final answer
{"role": "assistant", "content": "There are 4 customers who pay in EUR."},] ```
```

This is the agents short-term memory. For the next question, the LLM can see it already knows the tables and the customers schema - so a smart agent will skip the redundant tool calls and go straight to writing SQL.

## Key Takeaways

1. Tools are plain Python functions. The MCP decorator handles schema generation and transport. You focus on the logic.
  2. The server is LLM-agnostic. It exposes tools via MCP. The client converts them to whatever format the LLM needs. Swap OpenAI for Claude or Gemini - the server doesn't change.
  3. The ReAct loop is the agent. It's a loop that calls the LLM, checks for tool calls, executes them, and feeds results back. The LLM controls the flow - the code just follows instructions.
  4. Safety is layered. The system prompt tells the LLM the rules. The tool blocks forbidden keywords. The tool validates table names. Defense-in-depth - no single layer is the whole story.
  5. MCP is just plumbing. The protocol handles serialization, transport, and tool discovery. You write functions, decorate them, and everything works. That's the point.
- To run this code for real, see the [in the companion repository](#).